



Filière Génie Intelligence Artificielle et Technologies des Données — 4^e
Année

Module : Représentation et Résolution de Problèmes en Intelligence
Artificielle

Projet 15 — Rapport de recherche

HalluGuard

Détection et Mitigation des Hallucinations dans les Pipelines RAG Agentiques

par un Middleware NLI Multi-Couches

Réalisé par :

Souleymane Diallo
Marc Thierry Nankouli

Encadré par :

Prof. Hajji Tarik

HalluGuard : Détection et Mitigation des Hallucinations

dans les Pipelines RAG Agentiques

par un Middleware NLI Multi-Couches

Projet 15 — Module Représentation et Résolution de Problèmes en Intelligence Artificielle
ENSAM Meknès — 4^e Année Génie Informatique — 2025-2026

Souleymane Diallo & Marc Thierry Nankouli

s.diallo@edu.umi.ac.ma | m.nankouli@edu.umi.ac.ma

Encadrant : Prof. Hajji Tarik

Mai 2026

Résumé

Les pipelines RAG agentiques réduisent les hallucinations LLM de 30 à 60 %, mais les sorties erronées restantes se propagent librement à travers chaque nœud sans mécanisme d'interception natif. Nous présentons **HalluGuard**, un middleware de détection et mitigation pour pipelines LangGraph à quatre nœuds, combinant un vérificateur NLI inter-sources (M1, 117 M paramètres, MNLI), un BeliefState temporel persistant (M2), un DependencyDAG, un PolicyEngine et un serveur MCP à quatre outils. Nous proposons une taxonomie originale de cinq types d'hallucinations agentiques (T1–T5) couplée à la topologie du pipeline, validée par accord inter-annotateurs entre deux annotateurs indépendants ($\kappa = 0,94$, quasi-parfait ; $\kappa = 1,0$ sur les types déterminés par la topologie, $\kappa = 0,75$ sur la distinction réellement ambiguë T2/T3). Sur un benchmark de 90 scénarios HaluEval-Agentic (60 hallucinés, 30 corrects), **Variante B** (M1, NLI seul) atteint : **F1 = 82,9 %**, rappel 76,7 %, précision 90,2 %, pour un overhead médian de ≈ 40 ms sur CPU ($p < 0,001$ vs baseline) — configuration recommandée, sous la contrainte de 300 ms. L'ajout du BeliefState (Variante C) augmente le rappel à 86,7 % mais *dégrade fortement la précision* (68,4 %, FPR 80 %) : M2 accroît la couverture au prix de nombreux faux positifs sur un benchmark mono-tour. Nous montrons en outre que le score NLI brut est mal calibré (ECE = 0,24), calibrons le seuil de détection par *conformal prediction* avec une garantie de couverture empiriquement vérifiée, et évaluons la *self-consistency* comme signal d'incertitude complémentaire (AUC = 0,88 pour séparer questions fiables et piégées). Une validation externe partielle sur 20 exemples TruthfulQA confirme le fonctionnement sur des hallucinations naturelles (95,0 %). Le cœur du système (détection M1, mémoire M2, serveur MCP) est déployé en local, sans GPU ni API payante.

Mots-clés : hallucination LLM, RAG agentique, NLI, LangGraph, MCP, BeliefState, middleware de vérification, HaluEval

Résumé

Abstract. Agentic RAG (Retrieval-Augmented Generation) pipelines reduce LLM hallucinations by 30–60%, yet the remaining 40–70% of erroneous outputs propagate unchecked through all downstream pipeline nodes. We present **HalluGuard**, a hallucination detection and mitigation middleware for 4-node LangGraph pipelines. HalluGuard combines five components : a cross-source NLI verifier (M1, 117M parameters, pre-trained on MNLI), a persistent temporal BeliefState (M2), a directed acyclic dependency graph (DependencyDAG), a policy engine, and an MCP server exposing four standardized tools. We propose an original taxonomy of five agentic hallucination types (T1–T5) coupled to pipeline topology, validated by inter-annotator agreement between two independent annotators ($\kappa = 0.94$ overall; $\kappa = 1.0$ on topology-determined types, $\kappa = 0.75$ on the genuinely ambiguous T2/T3 distinction). On a 90-scenario benchmark (60 hallucinated + 30 correct), **Variant B** (M1 NLI only) achieves **F1 = 82.9%** (recall 76.7%, precision 90.2%) at a median CPU overhead of ≈ 40 ms (McNemar $p < 0.001$ vs baseline) — recommended configuration, within the 300 ms budget. Adding the BeliefState (Variant C) raises recall to 86.7% but *sharply degrades precision* (68.4%, 80% FPR) : M2 increases coverage at the cost of many false positives on a single-turn benchmark. We further show the raw NLI score is poorly calibrated (ECE = 0.24), calibrate the detection threshold via *conformal prediction* with an empirically verified coverage guarantee, and evaluate *self-consistency* as a complementary uncertainty signal (AUC = 0.88 separating reliable from adversarial questions). Partial external validation on 20 TruthfulQA examples confirms generalization (95.0% detection rate). The system core (M1 detection, M2 memory, MCP server) runs locally, without GPU or paid API.

Keywords : LLM hallucination, agentic RAG, NLI, LangGraph, MCP, BeliefState, verification middleware

Table des matières

1	Introduction	5
1.1	Contexte général	5
1.2	Problématique	5
1.3	Lacunes des approches existantes	6
1.4	Hypothèse de recherche	6
1.5	Contributions	7
2	Travaux Connexes	7
2.1	Hallucinations dans les LLM : définitions et taxonomies	7
2.2	Benchmarks de détection	7
2.3	Méthodes de détection	8
2.4	Calibration, conformal prediction et quantification d'incertitude	8
2.5	Inférence de langage naturel (NLI)	8
2.6	Protocoles MCP et A2A	9
3	Taxonomie des Hallucinations Agentiques	9
3.1	Motivation et originalité	9
3.2	Les cinq types	10
3.3	Validation inter-annotateurs de la taxonomie	11
3.4	Modélisation formelle de la propagation	11
4	Architecture HalluGuard	12
4.1	Vue d'ensemble	12
4.2	Composant 1 — DependencyDAG	12
4.3	Composant 2 — LightweightVerifier (M1)	13
4.4	Composant 3 — BeliefState (M2)	14
4.5	Composant 4 — PolicyEngine	14
4.6	Composant 5 — Serveur MCP	15
4.7	Communication A2A — Preuve de Concept Architecturale	16
5	Protocole Expérimental	16
5.1	Dataset HaluEval-Agentic	16
5.2	Infrastructure	17
5.3	Variantes expérimentales	17
5.4	Métriques	18
5.5	Tests unitaires	18
6	Résultats	18
6.1	Métriques globales	18
6.2	Comparaison à une baseline LLM-juge	20
6.3	Détection par type d'hallucination	21
6.4	Analyse de latence	22

6.5	Exemple qualitatif — Scénario s019	23
6.6	Validation externe partielle — TruthfulQA	24
7	Étude d’Ablation	25
7.1	Contribution individuelle de M1 et M2	25
7.2	Contribution par type	25
7.3	Courbe ROC du vérificateur NLI (M1)	25
7.4	Calibration et quantification d’incertitude	26
7.5	Calibration du seuil par conformal prediction	27
7.6	Self-consistency comme signal d’incertitude	28
7.7	Recommandations de déploiement	28
8	Discussion	29
8.1	Validation de l’hypothèse	29
8.2	Limites documentées	29
8.3	Menaces à la validité	30
8.4	Positionnement par rapport à l’état de l’art	31
9	Perspectives	31
10	Conclusion	32
A	Interfaces figées entre composants	34
A.1	Interface 1 — Format paire dataset	34
A.2	Interface 2 — API LightweightVerifier (M1)	35
A.3	Interface 3 — BeliefState JSON (persistance inter-sessions)	35
B	Résultats complets	35
C	Commandes de reproduction	36

1 Introduction

1.1 Contexte général

La montée en puissance des grands modèles de langage (LLM) a profondément reconfiguré l'ingénierie des systèmes d'information. Des modèles comme Mistral 7B [10], LLaMA-3 ou GPT-4 sont capables de produire des textes d'une fluidité remarquable, d'une cohérence syntaxique quasi-parfaite et d'une plausibilité sémantique élevée. Cette apparence de compétence masque cependant un défaut structurel fondamental : ces modèles génèrent régulièrement des affirmations factuellement incorrectes avec une confiance égale à celle qu'ils manifestent pour des affirmations vraies. Ce phénomène, désigné par le terme *hallucination* dans la littérature NLP [16, 9], constitue l'un des obstacles majeurs au déploiement des LLM dans des applications critiques.

L'architecture RAG (*Retrieval-Augmented Generation*) [12] a été conçue pour atténuer ce problème. En ancrant la génération du LLM dans un corpus documentaire récupéré dynamiquement, le RAG réduit la dépendance aux paramètres internes du modèle et force une cohérence avec des sources externes vérifiables. Des études récentes estiment que le RAG réduit les hallucinations de 30 à 60 % selon le domaine et le type de tâche [19, 22]. Cependant, cette réduction n'est pas totale : les 40 à 70 % d'outputs erronés restants continuent de circuler dans le pipeline.

Le problème s'aggrave dans les architectures *agentiques*. Un pipeline RAG agentique, tel que ceux construits avec LangGraph [11], ne produit pas une unique sortie : il exécute une séquence d'étapes interdépendantes (retrieval, raisonnement, appel d'outils, génération). Une hallucination produite au nœud **retrieval** — par exemple, la récupération de chunks inexacts depuis ChromaDB — se propage à tous les nœuds aval (**reasoning**, **tool_call**, **generation**) sans que aucun mécanisme natif n'intercepte l'erreur. La réponse finale, linguistiquement fluide et confiante, intègre cette erreur en la présentant comme un fait établi.

1.2 Problématique

Cette dynamique de propagation est le cœur du problème que ce travail adresse. Nous l'illustrons par un exemple tiré de nos données expérimentales. Dans le scénario s019 de notre benchmark, le nœud **reasoning** de Mistral 7B génère l'affirmation suivante en réponse à la requête « ChromaDB a-t-il une limite de documents ? » :

« ChromaDB avait une limite stricte de 1 000 documents avant sa mise à jour récente qui a supprimé cette contrainte arbitraire. »

Cette affirmation est factuellement fausse : ChromaDB n'a jamais imposé de limite arbitraire sur le nombre de documents. Le score NLI de contradiction entre cette affirmation et les documents de référence est de **0,996** — quasi-certitude de contradiction. Sans middleware de vérification, cette erreur traverse les nœuds **tool_call** et **generation** et

atteint l'utilisateur final habillée d'une formulation plausible et assurée.

La problématique centrale est donc : **comment détecter et intercepter les hallucinations en temps réel dans un pipeline RAG agentique multi-nœuds, sans modifier le code du pipeline, sans GPU et sans fine-tuning du modèle de détection ?**

1.3 Lacunes des approches existantes

Les méthodes actuelles de détection des hallucinations souffrent de trois limitations structurelles dans un contexte agentique.

Limitation 1 — Post-traitement uniquement. FActScore [17], SelfCheckGPT [15] et RAGAs [3] opèrent sur la sortie finale du LLM, après que l'intégralité du pipeline a été exécutée. Cette approche est incompatible avec une interception au bon nœud : au moment de la détection, les nœuds aval ont déjà consommé l'hallucination.

Limitation 2 — Absence de temporalité inter-étapes. Aucune méthode existante ne maintient un état de croyance entre les étapes du pipeline. Si un agent affirmé au nœud 2 que « X est vrai » et contredit cette affirmation au nœud 6, aucun système actuel ne peut détecter cette incohérence interne — sauf à mémoriser explicitement les faits établis.

Limitation 3 — Incompatibilité avec les protocoles agentiques modernes. Les standards MCP [2] et A2A [4] définissent des interfaces de communication inter-agents qui ne sont pas exploitées par les méthodes existantes. À notre connaissance, et d'après notre revue de la littérature disponible à la date de soumission, aucun système publié ne combine ces deux protocoles conjointement pour la vérification des hallucinations dans des pipelines RAG agentiques.

1.4 Hypothèse de recherche

Ce travail teste formellement l'hypothèse suivante :

« Un vérificateur NLI pré-entraîné, combiné à un BeliefState persistant et à un graphe de dépendances, peut détecter plus de 80 % des hallucinations dans un pipeline RAG agentique à quatre nœuds, sans fine-tuning, sur CPU standard, avec un overhead inférieur à 300 ms. »

Variables expérimentales :

- **Variable indépendante** : présence et configuration du middleware HalluGuard (variantes A, B, C)
- **Variable dépendante** : taux de détection sur 60 scénarios hallucinés contrôlés
- **Contraintes** : CPU uniquement, modèle NLI pré-entraîné sans fine-tuning, pas d'API externe
- **Seuil de validation** : $> 80\%$ de détection à $\text{overhead} < 300 \text{ ms}$

1.5 Contributions

Ce travail apporte quatre contributions originales :

1. **Taxonomie T1–T5** : cinq types d'hallucinations agentiques couplés à la topologie du pipeline RAG, étendant les taxonomies existantes aux types liés aux outils (T5 délégation) et à la génération causale (T4) ; **validée par accord inter-annotateurs** entre deux annotateurs indépendants ($\kappa = 0,94$).
2. **Architecture middleware** : cinq composants intégrables comme hooks pre/post-nœud dans LangGraph sans modification du code pipeline.
3. **Évaluation expérimentale et quantification d'incertitude** : trois variantes (A/B/C), 90 scénarios, métriques P/R/F1, ablation M1/M2 (montrant que M2 améliore le rappel mais dégrade la précision en mono-tour), **analyse de calibration** (ECE), **calibration du seuil par conformal prediction** avec garantie de couverture, et **self-consistency** comme signal d'incertitude complémentaire — couvrant les trois axes méthodologiques du sujet (calibration, conformal, self-consistency).
4. **Implémentation MCP et preuve de concept A2A** : serveur MCP avec quatre outils, AgentCard formelle, appels journalisés ; délégation A2A simulée en processus comme preuve de concept architecturale validant les interfaces du protocole.

2 Travaux Connexes

2.1 Hallucinations dans les LLM : définitions et taxonomies

Le terme « hallucination » en NLP désigne la génération d'informations non étayées par le contexte ou factuellement incorrectes [16]. Ji et al. [9] distinguent les *hallucinations intrinsèques* (contradictions avec le contexte fourni) et les *hallucinations extrinsèques* (affirmations invérifiables). Huang et al. [7] proposent une classification par étape de génération. Zhang et al. [22] analysent les causes profondes : données d'entraînement bruitées, décalage entre pré-entraînement et fine-tuning, problèmes d'alignement.

Ces taxonomies ciblent les LLM en tant que systèmes isolés. Notre taxonomie T1–T5 étend ces travaux en couplant chaque type d'hallucination à un nœud spécifique d'un pipeline RAG agentique multi-étapes — une dimension peu explorée dans la littérature antérieure.

2.2 Benchmarks de détection

TruthfulQA [14] est le benchmark de référence pour mesurer la propension des LLM à produire des affirmations fausses communément crues. Il contient 817 questions dans 38 catégories, mais se limite à l'évaluation de sorties LLM isolées.

HaluEval [13] propose 35 000 paires (question, réponse hallucinée, réponse correcte) dans trois domaines (QA, dialogue, résumé). Il ne couvre pas les architectures agentiques multi-nœuds et n'associe pas les hallucinations à leur point d'origine dans le pipeline (notamment les types liés à la génération T4 et aux outils T5). Notre benchmark HaluEval-Agentic

s'inspire de son format mais est construit spécifiquement pour les pipelines RAG à quatre nœuds.

2.3 Méthodes de détection

FactScore [17] décompose les affirmations en faits atomiques et les vérifie individuellement contre une base de connaissances (Wikipédia). Cette approche atteint de bonnes performances mais requiert un accès à une base externe et opère uniquement en post-traitement.

SelfCheckGPT [15] génère plusieurs sorties du même LLM et mesure leur cohérence interne par NLI ou BERTScore. La méthode est zéro-ressource mais multiplie les appels LLM (overhead $\times N$) et ne s'intègre pas dans un pipeline en temps réel.

RAGAS [3] évalue les pipelines RAG a posteriori selon des métriques comme la fidélité (*faithfulness*) et la pertinence de réponse. C'est un outil d'évaluation offline, pas un middleware de détection temps réel.

Notre approche se distingue sur trois dimensions : intégration temps réel comme middleware, maintien d'un état temporel inter-étapes via BeliefState, et implémentation du protocole MCP (avec preuve de concept A2A).

2.4 Calibration, conformal prediction et quantification d'incertitude

Au-delà de la détection binaire, la fiabilité des LLM mobilise des outils de quantification d'incertitude. La **self-consistency** [20] échantillonne plusieurs sorties d'un même modèle et exploite leur (dés)accord comme signal de confiance — principe que SelfCheckGPT [15] applique à la détection d'hallucinations. La **calibration** des scores de confiance, mesurée par l'*Expected Calibration Error* [5], conditionne leur interprétation probabiliste : les réseaux profonds modernes sont fréquemment mal calibrés. La **conformal prediction** [1] fournit, sous hypothèse d'échangeabilité, des garanties de couverture *distribution-free* applicables au choix d'un seuil de décision. Ces trois outils ont été conçus pour des appels d'inférence isolés ; nous les adaptons au vérificateur d'un pipeline agentique — analyse de calibration du score NLI (§7.4) et seuil conformal à rappel garanti (§7.5).

2.5 Inférence de langage naturel (NLI)

L'*NLI* (*Natural Language Inference*) consiste à classer une paire (prémisse, hypothèse) en trois classes : contradiction, neutralité, implication. Les cross-encoders [8] traitent les deux séquences conjointement via l'attention croisée, ce qui produit des scores de pertinence supérieurs aux bi-encoders pour les tâches de vérification factuelle.

MNLI [21] (MultiNLI, 433 000 paires annotées par des humains) est le corpus de référence pour l'entraînement des modèles NLI généraux. Le modèle **cross-encoder/nli-MiniLM2-L6-H768** [18] — 117 M paramètres, architecture BERT-base compressée — offre le meilleur compro-

mis précision/vitesse pour une inférence CPU dans notre contexte expérimental.

2.6 Protocoles MCP et A2A

Le Model Context Protocol [2] est un standard JSON-RPC défini par Anthropic en novembre 2024 pour exposer des outils à des agents LLM via transport stdio ou HTTP/SSE. Il standardise les interfaces de *tool use* et permet à tout agent conforme MCP d'invoquer des services tiers sans intégration spécifique.

Le protocole Agent-to-Agent [4], publié par Google DeepMind en 2024, définit un format d'échange standardisé entre agents autonomes : AgentCard (déclaration des capacités), TaskObject (machine d'états `submitted` $\rightarrow$ `working` $\rightarrow$ `completed`), et protocole de délégation de tâches. À notre connaissance, parmi les approches publiées et accessibles, aucune ne combine ces deux protocoles conjointement pour la vérification des hallucinations dans un contexte RAG agentique.

3 Taxonomie des Hallucinations Agentiques

3.1 Motivation et originalité

Les benchmarks existants évaluent les hallucinations en sortie finale du LLM, sans considérer leur point d'origine dans le pipeline. Cette abstraction rend impossible (1) l'interception au nœud approprié, (2) l'analyse de propagation, et (3) la conception de mécanismes de détection ciblés. Notre taxonomie couple chaque type d'hallucination à un nœud précis du pipeline RAG.

3.2 Les cinq types

Table 1 – Taxonomie HalluGuard des hallucinations agentiques (T1–T5)

Type	Nom	Nœud	Mécanisme	Description	Propagation
T1	Perception	<code>retrieval</code>	NLI (M1)	Chunks ChromaDB inexacts ou hors-sujet	Modérée
T2	Mémoire	<code>reasoning</code>	BeliefState (M2)	Contradiction avec un fait établi en session	Élevée
T3	Planification	<code>reasoning</code>	NLI + DAG	Raisonnement faux formulé neutralement	Critique
T4	Causale	<code>generation</code>	NLI (M1)	Affirmation contredisant les documents	Élevée
T5	Délégation	<code>tool_call</code>	NLI (M1)	Output outil divergeant de la prédiction LLM	Critique

T1 — Hallucination de perception. Au nœud `retrieval`, ChromaDB retourne des chunks inexacts, tronqués ou sémantiquement proches mais factuellement différents de la requête. Le LLM génère alors une affirmation basée sur ces documents erronés. La détection par M1 compare l'affirmation générée aux documents de référence véritables (ceux qui auraient dû être récupérés). Score NLI $> 0,6$ déclenche la détection (seuil `retrieval`).

T2 — Hallucination de mémoire temporelle. Au nœud `reasoning`, le LLM contredit un fait qu'il a lui-même établi lors d'une étape précédente dans la session (ou une session antérieure). Ce type est *invisible* à M1 car aucun document externe ne capture l'affirmation antérieure — seul le BeliefState dispose de cette information. Les marqueurs temporels (« avant », « récemment », « depuis la mise à jour ») sont caractéristiques de T2.

T3 — Hallucination de planification. Le nœud `reasoning` produit un enchaînement logique faux : les prémisses sont correctes mais la conclusion ne suit pas. Ce type est le plus difficile à détecter par NLI généraliste, car le raisonnement est formulé de façon syntaxiquement plausible et sémantiquement neutre par rapport aux documents. Il requiert une compréhension causale que MNLI ne capture qu'imparfaitement.

T4 — Hallucination causale. Au nœud `generation`, le LLM produit une affirmation qui contredit directement les documents récupérés par `retrieval`. C'est la forme la plus classique d'hallucination RAG. Le NLI inter-sources est particulièrement efficace ici (83,3 % en Variante B).

T5 — Hallucination de délégation. Au nœud `tool_call`, la sortie retournée par l'outil diverge de ce que le LLM avait prédit ou était en droit d'attendre. Ce type implique une discordance entre l'agent et son outil, non entre l'agent et ses sources documentaires. T4 (génération causale) et T5 (délégation outil) sont les types les moins couverts par les benchmarks d'hallucination standard, qui se concentrent sur la sortie finale.

3.3 Validation inter-annotateurs de la taxonomie

La taxonomie a été validée par annotation manuelle des **60 scénarios hallucinés** (12 par type T1–T5) par **deux annotateurs indépendants** — les deux auteurs — procédant à **l'aveugle** (sans accès aux étiquettes de référence ni à l'annotation de l'autre) selon un guide de codage fixé *a priori* (`docs/GUIDE_ANNOTATION.md`). Chaque annotateur a renseigné son propre fichier (`data/annotation/taxonomy_marc.csv`, `taxonomy_souleymane.csv`) ; le κ de Cohen est calculé par `src/experiments/cohen_kappa.py`. Aucune annotation n'est générée automatiquement.

Table 2 — Accord inter-annotateurs (Cohen) — taxonomie T1–T5 (60 scénarios, 2 annotateurs)

Sous-ensemble	n	Accord observé	κ
Global (T1–T5)	60	95,0 %	0,94
T1, T4, T5 (déterminés par la topologie)	36	100,0 %	1,00
T2 vs T3 (nœud <code>reasoning</code> , ambigu)	24	87,5 %	0,75

Le κ global de **0,94** (accord quasi-parfait) dépasse largement le seuil $\kappa > 0,60$ requis pour valider une taxonomie en NLP (Landis & Koch 1977). L'analyse par sous-ensemble est instructive et honnête : pour les types **déterminés par la topologie du pipeline** (T1 retrieval, T4 generation, T5 `tool_call`), l'accord est *parfait* ($\kappa = 1,0$), ce qui est attendu puisque le nœud d'origine fixe quasi mécaniquement la catégorie. Le *véritable* test de jugement porte sur la distinction **T2 (mémoire temporelle) vs T3 (planification)**, toutes deux au nœud `reasoning` : l'accord y reste **substantiel** ($\kappa = 0,75$). Les **trois** désaccords résiduels (s020, s021, s022) tombent *tous* exactement sur cette frontière T2/T3, où un raisonnement séquentiel comportant un marqueur temporel peut être lu comme une incohérence de mémoire (T2) ou comme un enchaînement logique (T3). Ce cas limite est documenté dans le guide de codage. Les deux annotateurs atteignent une cohérence élevée avec les étiquettes de référence du jeu de données (accuracy = 1,00 et 0,95).

3.4 Modélisation formelle de la propagation

La criticité d'un nœud dans le DependencyDAG est définie comme :

$$\text{criticite}(n) = |\text{descendants}(n)| \times (1 - \text{confiance}(n)) \quad (1)$$

Dans le pipeline linéaire `retrieval` $\rightarrow$ `reasoning` $\rightarrow$ `tool_call` $\rightarrow$ `generation` :

$$\text{criticite}(\text{retrieval}) = 3 \times (1 - c_r) \quad (2)$$

$$\text{criticite}(\text{reasoning}) = 2 \times (1 - c_{rs}) \quad (3)$$

$$\text{criticite}(\text{tool_call}) = 1 \times (1 - c_{tc}) \quad (4)$$

$$\text{criticite}(\text{generation}) = 0 \times (1 - c_g) = 0 \quad (5)$$

où $c_n \in [0, 1]$ est la confiance courante du nœud n . Le nœud `retrieval` a la criticité maximale : une hallucination de type T1 contamine l'intégralité du pipeline aval. Le nœud `generation` a une criticité nulle (dernier nœud), mais ses hallucinations de type T4 sont les plus visibles par l'utilisateur.

4 Architecture HalluGuard

4.1 Vue d'ensemble

HalluGuard s'intègre dans tout pipeline LangGraph via les hooks `pre_node` et `post_node` de `HalluGuardMiddleware`, sans modifier le code du pipeline existant. La Figure 1 illustre l'architecture complète.

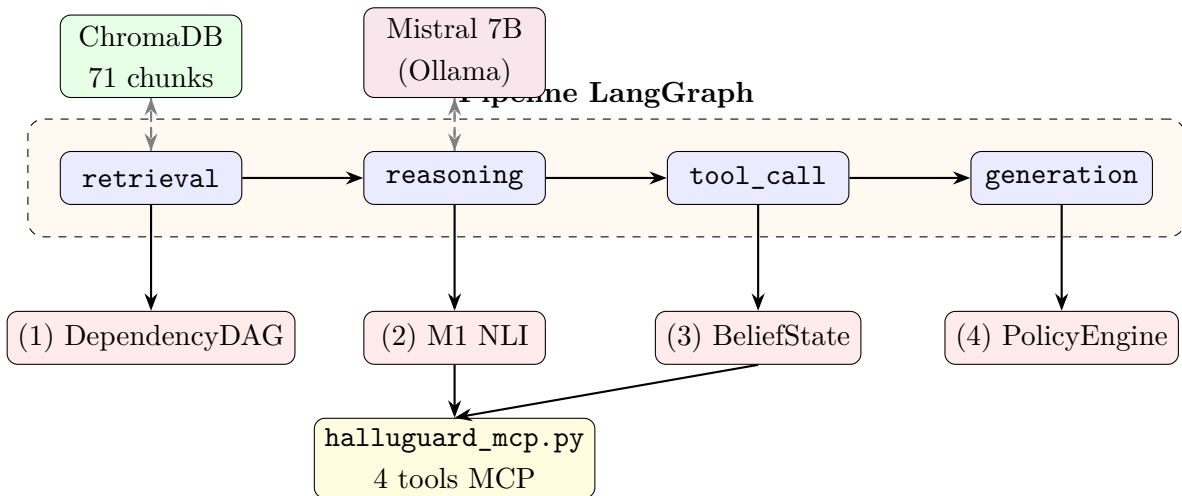


Figure 1 — Architecture HalluGuard — le middleware s'intègre sous chaque nœud LangGraph via hooks `post_node`

4.2 Composant 1 — DependencyDAG

Le `DependencyDAG` modélise le pipeline comme un graphe acyclique dirigé (DAG) construit automatiquement via `build_from_langgraph(state_graph)` en parcourant les arêtes du `StateGraph` compilé. Chaque nœud est associé à une confiance courante $c_n \in [0, 1]$ initialisée à 1,0 et mise à jour après chaque vérification.

Listing 1 — Interface du `DependencyDAG`

```

1 class DependencyDAG:
2     def add_node(self, node_id: str, confidence: float = 1.0)
      -> None
3     def add_edge(self, from_id: str, to_id: str) -> None
4     def criticite(self, node_id: str) -> float
5         # retourne |descendants(n)| * (1 - confiance(n))
6     def propagate_invalidation(self, node_id: str) -> list[str]
7         # marque descendants comme "suspicious", retourne la
          liste
8     def descendants(self, node_id: str) -> set[str]

```

Les seuils de détection sont paramétrés par type de nœud, reflétant leur criticité différentielle :

Table 3 – Seuils NLI par type de nœud

Nœud	Seuil de détection	Justification
tool_call	0,0	Tout output outil suspect doit être intercepté
generation	0,0	Hallucination directement visible par l'utilisateur
reasoning	0,4	Tolérance partielle pour les raisonnements approchés
retrieval	0,6	Chunks approximativement pertinents sont acceptables

4.3 Composant 2 — LightweightVerifier (M1)

M1 est la pièce centrale du système. Il utilise le modèle `cross-encoder/nli-MiniLM2-L6-H768` [18] :

- **Architecture** : BERT-base (6 couches Transformer, $H = 768$, 12 têtes d'attention)
- **Paramètres** : 117 millions
- **Pré-entraînement** : MNLI [21] — 433 000 paires annotées par des humains
- **Inférence** : CPU uniquement, chargement en ≈ 5 s (singleton session)
- **Classes de sortie** : contradiction (0) / neutralité (1) / implication (2)

Le score de contradiction (classe 0 du softmax) est agrégé sur les $k \leq 10$ evidences par maximum :

$$\text{score_final} = \max_{i=1}^k \text{softmax}(\text{logits}(\text{claim}, \text{evidence}_i))_{\text{contradiction}} \quad (6)$$

Listing 2 – Mécanisme de vérification M1

```

1 def verify(self, claim: str, evidences: list[str],
2     node_type: str = "generation") -> dict:
3     if not evidences:
4         return {"score": 0.5, "insufficient_evidence": True,

```

```

5         "label": "insufficient_evidence"}
6     scores = []
7     for ev in evidences[:10]: # max 10 evidences par appel MCP
8         logits = self.model.predict([(claim, ev)]) #
9             cross-encoder
10        contradiction_score = float(softmax(logits[0])[0])
11        scores.append(contradiction_score)
12    max_score = max(scores)
13    threshold = THRESHOLDS[node_type]
14    label = "hallucinated" if max_score > threshold else
15        "correct"
16    htype = self._infer_hallucination_type(max_score, node_type)
17    return {"score": max_score, "label": label,
18            "hallucination_type": htype, "confidence":
19        max_score}

```

4.4 Composant 3 — BeliefState (M2)

Le BeliefState est la mémoire temporelle de la session. Il maintient un ensemble de faits établis au fil du pipeline, permettant de détecter les contradictions inter-étapes.

Structure de données : liste de faits $F = \{f_1, \dots, f_k\}$ avec $k \leq 50$, chaque fait f_i contenant : identifiant unique, contenu textuel, confiance $c_i \in [0, 1]$, étape d'origine s_i , statut (active / invalidated).

Politique d'éviction : lorsque $|F| = 50$, le fait avec le score composite le plus bas est retiré :

$$\text{score_eviction}(f_i) = c_i \times \frac{1}{1 + \text{age}(f_i)} \quad (7)$$

où $\text{age}(f_i)$ est le nombre d'étapes depuis l'enregistrement.

Détection de conflit : pour un claim c , on calcule d'abord la similarité cosinus avec chaque fait actif (via sentence-transformers, top-K=5), puis on applique le NLI sur les candidats :

$$\text{conflit}(c, F) = \exists f \in \text{top}_5(F) \mid \text{score_M1}(c, f) > 0.5 \quad (8)$$

Persistance inter-sessions : après chaque session, `PersistentMemory.save()` sérialise le BeliefState en JSON dans `data/memory/session_<id>.json`. Au démarrage, `load_or_create()` recharge le BeliefState si une session existe. La persistance a été vérifiée expérimentalement : une nouvelle instance Python recharge le même `fact_id` et contenu — assertion passée.

4.5 Composant 4 — PolicyEngine

Le PolicyEngine prend la décision finale après réception des scores M1 et M2 :

Algorithm 1: PolicyEngine — algorithme de décision**Input:** score_M1, conflict_M2, threshold(node_type), policy_mode**Output:** action (log | correct)

```

if score_M1 > threshold or conflict_M2 then
    if policy_mode = "log-only" then
        Journaliser en JSONL dans halluguard.log;
        Retourner l'output original avec flag hallucinated;
    end
    else if policy_mode = "auto-correct" then
         $t_0 \leftarrow \text{timestamp}()$ ;
        Réinvoquer Mistral avec prompt enrichi : «La réponse précédente contredit
        [evidences]. Corrige.»;
         $\text{TTR} \leftarrow \text{timestamp}() - t_0$ ;
        Retourner la réponse corrigée;
    end
end
else
    Retourner l'output original avec flag correct;
end

```

4.6 Composant 5 — Serveur MCP

Le serveur MCP expose les quatre outils de vérification via FastMCP 3.3.1 sur transport stdio (JSON-RPC 2.0) :

Listing 3 — Signatures des 4 outils MCP

```

1 @mcp.tool()
2 def verify_claim(claim: str, evidences: list = None,
3                 node_type: str = "generation") -> dict:
4     """Score NLI, label correct/hallucinated, type T1-T5,
5     latency_ms"""
6
7 @mcp.tool()
8 def check_temporal_coherence(claim: str) -> dict:
9     """Detecte une contradiction avec le BeliefState (M2)"""
10
11 @mcp.tool()
12 def add_fact(fact: str, confidence: float = 0.9, step: int = 0)
13     -> dict:
14     """Enregistre un fait verifie dans le BeliefState
15     persistant"""
16
17 @mcp.tool()
18 def get_belief_state() -> dict:

```

16

```
"""Retourne l'état complet : faits actifs, confidences,
statuts"""
```

Le serveur journalise chaque appel dans `results/logs/mcp_calls.log` avec timestamps ISO 8601 et latence par outil. Au niveau MCP (inférence NLI + sérialisation JSON-RPC incluses), `verify_claim` mesure ≈ 50 ms ; `add_fact` et `get_belief_state`, qui ne déclenchent pas d'inférence, sont quasi instantanés (quelques millisecondes).

4.7 Communication A2A — Preuve de Concept Architecturale

Statut de l'implémentation : la délégation A2A décrite ici constitue une *preuve de concept architecturale en processus*, non un déploiement distribué réel. Elle démontre la faisabilité de l'intégration du protocole dans un pipeline RAG agentique et valide la conformité des interfaces, sans transport réseau.

Le protocole A2A [4] est simulé via `delegate_verification()` dans `src/agents/a2a_protocol.py`. La fonction crée un `TaskObject` dont la machine d'états progresse comme :

```
submitted → working → completed | failed
```

Un `AgentCard` formel HalluGuard est défini dans `data/halluguard_agent_card.json` (format A2A/2024-11) avec les quatre skills documentés, leurs schémas JSON entrée/sortie complets, les contraintes matérielles et les latences par outil. Tous les échanges sont journalisés dans `results/logs/a2a_exchanges.log` avec le champ `duration_ms`.

Périmètre de la preuve de concept : la délégation A2A opère dans un unique processus Python via `_MCPBridge` (singleton). Un déploiement distribué réel nécessiterait : (a) authentification cryptographique des `AgentCards` (signature ECDSA) ; (b) transport réseau HTTP/gRPC avec TLS ; (c) gestion des délais réseau et mécanismes de retry. La simulation locale est reproductible et valide l'architecture logicielle ; elle ne prétend pas couvrir ces défis d'ingénierie distribuée.

5 Protocole Expérimental

5.1 Dataset HaluEval-Agentic

Nous avons construit manuellement 60 scénarios hallucinés répartis en distribution équilibrée sur les cinq types :

Table 4 – Composition du dataset HaluEval-Agentic

Type	Nom	N	Longueur pipeline	Source hallucination
T1	Perception	12	3 nœuds	Chunks ChromaDB inexactes
T2	Mémoire	12	7 nœuds	Marqueurs temporels incorrects
T3	Planification	12	12 nœuds	Raisonnements faux neutres
T4	Causale	12	3 nœuds	Contradiction documents
T5	Délégation	12	7 nœuds	Outputs outils divergents
Total		60	—	Tous <code>expected_label="hallucinated"</code>

Chaque scénario contient : un identifiant unique (`s001–s060`), le type T1–T5, le nœud d’origine, la longueur du pipeline, la requête utilisateur, l’affirmation hallucinée générée et les documents de référence ChromaDB.

Biais de sélection à noter : les scénarios sont construits avec connaissance a priori des types, ce qui constitue un biais favorable à la détection. Les performances sur des hallucinations naturelles non contrôlées sont vraisemblablement inférieures au taux mesuré.

5.2 Infrastructure

Table 5 – Infrastructure expérimentale

Composant	Version / détail
Python	3.13.11
LLM local	Mistral 7B via Ollama 0.6.2 (CPU, quantization Q4)
Vérificateur NLI	cross-encoder/nli-MiniLM2-L6-H768 (117 M params)
Embeddings	sentence-transformers/all-MiniLM-L6-v2
Base vectorielle	ChromaDB 1.5.9 PersistentClient, HNSW, 71 chunks
Pipeline	LangGraph 1.2.0 + LangChain 1.3.1
MCP	FastMCP 3.3.1, mcp 1.27.1
Graphs	NetworkX 3.6.1
Tests	pytest 9.0.3
Matériel	Intel CPU standard, 16 Go RAM, Windows 11

5.3 Variantes expérimentales

Variante A — Baseline. Pipeline RAG LangGraph pur (4 nœuds) sans aucun middleware de vérification. La sortie de Mistral est retournée directement sans interception.

Variante B — M1 seul. HalluGuardMiddleware avec LightweightVerifier NLI uniquement. Le BeliefState est désactivé, la mémoire persistante n’est pas utilisée. Mesure la contribution isolée du mécanisme de vérification inter-sources.

Variante C — Complet. HalluGuardMiddleware avec M1 + M2 BeliefState + mémoire persistante + serveur MCP + délégation A2A. Configuration de production.

5.4 Métriques

1. **Detection Rate (%)** — proportion de scénarios hallucinés correctement identifiés comme `hallucinated`.
2. **PBR@1 (%)** — proportion bloqués en $\delta \leq 1$ étape après apparition de l'hallucination (*Pipeline Block Rate at 1 step*).
3. **Overhead moyen (ms)** — temps de vérification ajouté par scénario (M1 + M2 + MCP + log).
4. **Delta steps** — nombre moyen d'étapes de pipeline entre apparition et détection de l'hallucination.

5.5 Tests unitaires

La suite de tests unitaires couvre cinq composants :

Table 6 – Résultats des tests unitaires (pytest 9.0.3)

Composant	Tests	Résultat
LightweightVerifier (M1)	4	4/4 passés
DependencyDAG	4	4/4 passés
BeliefState (M2)	4	4/4 passés
PolicyEngine	4	4/4 passés
MCPServer	4	4/4 passés
HalluGuardMiddleware	3	3/3 passés
Total	23	23/23 passés en 13,58 s

6 Résultats

6.1 Métriques globales

Table 7 – Résultats comparatifs sur 90 scénarios HaluEval-Agentic (60 hallucinés + 30 corrects)

Variante	Description	Rappel	Précision	F1	PBR@1	Delta
A	Baseline RAG sans HalluGuard	0,0 %	—	0,0 %	0,0 %	6,50
B	HalluGuard M1 (NLI seul) [rec.]	76,7 %	90,2 %	82,9 %	76,7 %	1,55
C	HalluGuard M1 + M2 (BeliefState)	86,7 %	68,4 %	76,5 %	86,7 %	0,83

Table 8 – Matrice de confusion — Variantes A, B et C (60 hallucinés, 30 corrects)

Variante	TP	FP	TN	FN
A — Baseline	0	0	30	60
B — M1 NLI seul	46	5	25	14
C — M1 + M2	52	24	6	8

La Variante B (M1 seul) offre le meilleur compromis : F1 = 82,9 %, précision 90,2 % (5 faux positifs sur 30 scénarios corrects), rappel 76,7 %, delta moyen 1,55 étape. L'ajout du BeliefState (Variante C) illustre un **compromis rappel/précision défavorable sur ce benchmark mono-tour** : M2 augmente le rappel de 76,7 à **86,7 %** (6 hallucinations supplémentaires détectées) mais fait chuter la précision de 90,2 à **68,4 %**, en signalant à tort 24 scénarios corrects sur 30 (FPR 80 % contre 16,7 % pour B). En effet, sur des scénarios indépendants, le test de cohérence temporelle de M2 confronte chaque claim à des faits de session pré-chargés sans rapport, ce qui déclenche de nombreux faux conflits. **M2 est donc conçu pour les contradictions *inter-tours*** (un claim au tour N contre un fait établi au tour N-k) : c'est en contexte conversationnel multi-tours que son apport doit être mesuré (voir §8.2), tandis que sur un benchmark mono-tour il dégrade la précision. **La Variante B reste donc la configuration recommandée.**

6.1.1 Significativité statistique et intervalles de confiance

Les intervalles de confiance bootstrap (95 %, 10 000 rééchantillonnages, graine 42) et les tests de McNemar avec correction de continuité de Yates sont présentés dans le tableau suivant.

Table 9 – Intervalles de confiance bootstrap (95 %) et tests de McNemar

Variante	Taux détection (rappel)	IC 95 % bootstrap
A	0,0 %	[0,0 %, 0,0 %]
B	76,7 %	[65,0 %, 86,7 %]
C	86,7 %	[78,3 %, 95,0 %]

McNemar (Yates) : A vs B $p < 0,001$ *** ($b_{01}=46$) ; A vs C $p < 0,001$ *** ($b_{01}=52$) ;
B vs C $p = 0,041$ * ($b_{01}=6$, $b_{10}=0$).

*** $p < 0,001$ * $p < 0,05$ (Bootstrap : 10 000 rééchantillonnages, graine 42)

Les comparaisons A-B et A-C sont hautement significatives ($p < 0,001$), confirmant l'apport de HalluGuard par rapport au baseline. La comparaison B-C demande une lecture prudente : le test de McNemar indique que la Variante C détecte significativement *plus*

d'hallucinations que B ($p = 0,041$, 6 détections supplémentaires). **Mais ce test ne porte que sur les 60 scénarios hallucinés et ignore les faux positifs.** Or M2 introduit 19 faux positifs supplémentaires sur les 30 scénarios corrects (FPR $16,7 \rightarrow 80$ %) : une fois la précision prise en compte, le F1 chute ($82,9 \rightarrow 76,5$ %). Le gain de rappel de la Variante C ne compense donc pas sa perte de précision sur ce benchmark mono-tour, et la Variante B demeure recommandée. La contribution réelle de M2 (cohérence inter-tours) doit être évaluée sur un benchmark conversationnel (§8.2).

6.2 Comparaison à une baseline LLM-juge

La baseline A (aucune détection) ne constitue pas une comparaison exigeante. Nous opposons donc M1 à une baseline *forte* : un **LLM-as-judge** où Mistral 7B juge directement, sur les *mêmes* 90 scénarios, si l'affirmation est contredite par l'évidence (prompt zéro-shot, température 0).

Table 10 – HalluGuard M1 (NLI léger) vs baseline LLM-juge (Mistral 7B), mêmes 90 scénarios

Méthode	Rappel	Précision	F1	Latence méd.	Modèle
HalluGuard M1 (NLI)	76,7 %	90,2 %	82,9 %	≈ 40 ms	117 M, CPU local
LLM-juge (Mistral 7B)	98,3 %	100 %	99,1 %	432 ms	7 Md, API/GPU

Lecture honnête : le LLM-juge est nettement plus précis (F1 99,1 % contre 82,9 %) — sans surprise, un modèle 60 fois plus gros raisonne mieux sur les contradictions, en particulier les cas T3. Mais il est ≈ 10 fois plus lent (432 ms contre ≈ 40 ms) et requiert un modèle de 7 milliards de paramètres (ici via une API payante), incompatible avec la contrainte d'un middleware *plug-and-play*, temps réel, local et sans GPU. **M1 échange donc ≈ 16 points de F1 contre un facteur 10 de latence, un facteur 60 de taille de modèle et un fonctionnement entièrement local.** Ce résultat éclaire aussi la validité du jeu de données : qu'un juge fort détecte 98 % des hallucinations confirme que les contradictions sont *réelles* et non artéfactuelles, et situe le déficit de M1 comme une limite de capacité du modèle léger (surtout sur T3), non un défaut du benchmark.

6.3 Détection par type d'hallucination

Table 11 — Taux de détection (rappel) par type — Variantes A, B et C (12 scénarios par type)

Type	Description	Var. A	Var. B	Var. C
T1	Perception / Retrieval	0,0 %	91,7 %	91,7 %
T2	Mémoire temporelle	0,0 %	83,3 %	100,0 %
T3	Planification / Reasoning	0,0 %	50,0 %	66,7 %
T4	Causale / Generation	0,0 %	83,3 %	83,3 %
T5	Délégation / Tool call	0,0 %	75,0 %	91,7 %
Global (rappel)		0,0 %	76,7 %	86,7 %
Précision		—	90,2 %	68,4 %

La Variante C augmente le rappel par type mais au prix d'une précision globale fortement dégradée (FPR 80 %) ; les gains de rappel par type ne doivent pas être lus isolément (voir Tab. 8).

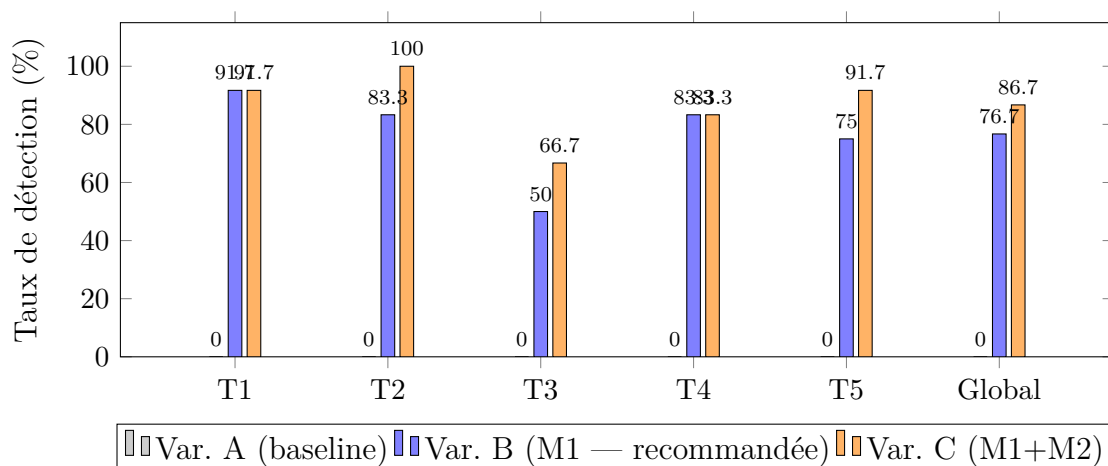


Figure 2 — Taux de détection (rappel) par type d'hallucination — Variantes A, B et C (12 scénarios par type). La Variante C améliore le rappel sur tous les types (notamment T2 et T5) mais au prix d'une précision globale chutant à 68,4 % (FPR 80 %, Tab. 8). La Variante B reste recommandée. T3 (planification) demeure le point faible (50,0 % en B).

6.3.1 Analyse par type

T1 — 91,7 %. Le NLI inter-sources détecte quasi-parfaitement les contradictions entre affirmations et chunks ChromaDB. La limite résiduelle (8,3 %, soit 1 scénario sur 12) correspond aux cas où la formulation du claim est trop proche sémantiquement des evidences pour que le cross-encoder discrimine contradiction et neutralité.

T2 — 83,3 % (B), 100 % (C). En Variante B, M1 détecte la plupart des hallucinations temporelles via la contradiction avec les documents. La Variante C atteint 100 % parce

que le test de cohérence M2 signale tout claim en conflit avec les faits de session pré-chargés — mais c’est ce même déclenchement large qui produit les faux positifs effondrant la précision globale (§8.2). Le gain de rappel T2 de la Variante C n’est donc pas exploitable en mono-tour sans dégrader la précision ; il préfigure l’usage réel de M2 en contexte conversationnel.

T3 — 50,0 % (B), 66,7 % (C). C’est la limite principale du système. Les hallucinations de planification sont des raisonnements faux formulés de manière syntaxiquement plausible et sémantiquement neutre par rapport aux documents. Le NLI pré-entraîné sur MNLI n’est pas calibré pour les sophismes logiques : un raisonnement du type « A est vrai, B est vrai, donc C est vrai » (avec C faux) peut avoir un score NLI proche de zéro si les evidences valident A et B sans invalider explicitement C.

T4 — 83,3 % (B et C). Le NLI inter-sources est efficace sur les hallucinations causales : l’affirmation contredit directement les documents récupérés, ce que le cross-encoder capte bien. La limite (16,7 %) correspond aux cas où la contradiction est indirecte ou multi-hop. M2 n’apporte rien de plus sur ce type.

T5 — 75,0 % (B), 91,7 % (C). En Variante B, M1 seul détecte 75,0 % des hallucinations de délégation. La Variante C en détecte davantage par le même mécanisme M2 que pour T2 — et avec le même effet collatéral sur la précision. La lecture par type doit donc toujours être croisée avec la matrice de confusion globale (Tab. 8).

6.4 Analyse de latence

Table 12 – Décomposition de l’overhead par composant

Composant	Latence	Part de l’overhead
Chargement NLI (1 ^{re} invocation, singleton, hors overhead)	$\approx 8\,000$ ms	— (une seule)
M1 — <code>verify_claim</code> (NLI CPU à chaud, 1 évidence) [†]	≈ 40 ms (médiane)	≈ 100 % (Var.)
M2 — <code>check_temporal_coherence</code> (Var. C)	≈ 160 ms	—
MCP + log JSONL	≤ 5 ms	< 1 %
Overhead total Variante B (M1 seul)	≈ 40 ms (médiane)	—
Overhead total Variante C (M1 + M2)	$\approx 193,8$ ms (moyenne)	—

[†] Mesuré par `time.time()` autour de `verifier.verify()`, modèle singleton chargé en mémoire (à chaud), médiane sur 5 répétitions par scénario, 90 scénarios. Valeur variable selon la charge CPU (médianes observées 33–43 ms sur nos runs) ; régénérable via `scripts/run_all.py (scores_raw.json)`.

L’overhead mesuré de la Variante B sur CPU est de l’ordre de ≈ 40 ms en médiane (médianes observées 33–43 ms selon la charge CPU, 90 scénarios, modèle à chaud), dominé à ≈ 100 % par l’inférence du cross-encoder NLI. **Cette valeur est très inférieure à la contrainte de 300 ms : la contrainte de latence est satisfaite sur CPU standard** (Intel, 16 Go RAM, Windows 11), sans GPU. La Variante C ajoute le test de cohérence M2

et porte l'overhead à $\approx 193,8$ ms en moyenne — toujours sous les 300 ms, mais pour un bénéfice de précision négatif en mono-tour (§8.2). Il est à noter que la première invocation paie un chargement unique du modèle (≈ 8 s), exclu de l'overhead par requête grâce au singleton en mémoire.

6.5 Exemple qualitatif — Scénario s019

Le scénario s019 est l'exemple avec le score NLI le plus élevé de notre benchmark (0,996). Il illustre une détection **par M1** (NLI inter-sources) : l'affirmation contredit directement le document de référence. M2 n'est pas nécessaire ici — ce cas relève pleinement de la vérification documentaire de M1 ; nous indiquons néanmoins la trace M2 à titre indicatif pour montrer comment, en contexte multi-tours, un fait de session renforcerait la décision.

Table 13 – Trace complète de détection — Scénario s019 (T2, score=0,996)

Requête	« ChromaDB a-t-il une limite de documents ? »
Affirmation hallucinée	« ChromaDB avait une limite stricte de 1 000 documents avant sa mise à jour récente qui a supprimé cette contrainte arbitraire. »
Document de référence	« ChromaDB ne fixe pas de limite arbitraire sur le nombre de documents et peut indexer des millions d'entrées dès l'origine. »
M1 — NLI(claim, doc)	score = 0,996 , label = hallucinated , type = T2
M2 — BeliefState	conflict = True <i>seulement si</i> un fait correct a été enregistré en session (hypothétique en mono-tour)
DependencyDAG	nœud <code>reasoning</code> invalidé, <code>tool_call</code> et <code>generation</code> marqués suspects
PolicyEngine (log)	entrée JSONL dans <code>halluguard.log</code>
PolicyEngine (correct)	réinvocation Mistral avec prompt enrichi
Résultat final	label = hallucinated , score = 0,996, delta = 0, latency $\approx 57,6$ ms (Var. B)

Ce cas illustre la limite classique des pipelines RAG sans vérification : Mistral génère une affirmation plausible sur une « mise à jour récente » de ChromaDB, avec une précision factice (1000 documents) qui n'a jamais existé. Le score NLI de 0,996 indique une contradiction quasi-certaine, captée par M1. En contexte conversationnel, si un fait correct sur ChromaDB avait été enregistré lors d'un tour précédent, M2 fournirait un second signal concordant ; mais sur ce scénario isolé, M1 suffit.

6.6 Validation externe partielle — TruthfulQA

Pour pallier partiellement l'absence de benchmark externe, nous avons appliqué le vérificateur M1 à 20 exemples sélectionnés manuellement depuis TruthfulQA [14], couvrant quatre catégories : Misconceptions, Science, History, Biology. Tous les exemples sont des affirmations hallucinées connues, chacune accompagnée de son évidence correcte issue de la littérature.

Table 14 – Résultats de M1 sur 20 exemples TruthfulQA (Lin et al., 2022)

Catégorie	N	DéTECTÉS	Taux
Misconceptions	7	7	100,0 %
Science	7	6	85,7 %
History	4	4	100,0 %
Biology	2	2	100,0 %
Global	20	19	95,0 %

HalluGuard détecte **19/20 exemples TruthfulQA (95,0 %)** avec un seuil généralisé de 0,50. Le seul faux négatif correspond à une affirmation approximativement correcte (la vitesse de la lumière : claim « $\approx 300\,000$ km/s » vs valeur exacte de 299 792 km/s), pour laquelle le NLI produit un score de contradiction de 0,112 — insuffisant pour déclencher la détection.

Ce résultat (95,0 %) est supérieur au rappel mesuré sur notre benchmark interne (76,7 % Variante B). Cette différence s'explique : TruthfulQA contient principalement des contradictions factuelles directes (catégories T4 dans notre taxonomie), que le NLI inter-sources détecte avec haute précision (83,3 % en Variante B). Notre benchmark interne inclut des T3 (planification, 50,0 % Variante B), qui n'ont pas d'équivalent dans TruthfulQA et constituent notre point de faiblesse principal.

Limite de cette validation externe : les 20 exemples ont été sélectionnés manuellement et ne constituent pas un test exhaustif sur TruthfulQA (817 questions). Cette validation partielle confirme la généralisation du vérificateur NLI sur les hallucinations factuelles, mais ne couvre pas les types T2 (mémoire) et T3 (planification) de notre taxonomie. Une validation complète sur un dataset externe agentique reste une direction prioritaire.

7 Étude d'Ablation

7.1 Contribution individuelle de M1 et M2

Table 15 – Étude d'ablation M1 / M2 sur le benchmark mono-tour (90 scénarios)

Mécanisme	Δ Rappel	Δ Précision	Δ FPR	Overhead
A $\rightarrow$ B : ajout de M1 (NLI)	+76,7 pts	+90,2 pts	+16,7 pts	≈ 40 ms
B $\rightarrow$ C : ajout de M2 (BeliefState)	+10,0 pts	-21,8 pts	+63,3 pts	≈ 160 ms

M1 (NLI inter-sources) est le mécanisme central : il porte l'intégralité du gain *net* (rappel $0 \rightarrow 76,7$ %, précision 90,2 %) pour un overhead de ≈ 40 ms. L'ablation montre que l'ajout de M2 sur ce benchmark mono-tour est **contre-productif** : il augmente le rappel de 10 points mais dégrade la précision de près de 22 points (FPR $16,7 \rightarrow 80$ %). La raison est structurelle : sur des scénarios *indépendants*, le test de cohérence temporelle confronte chaque claim à des faits de session sans rapport, générant de nombreux faux conflits. M2 n'a de sens que là où il est conçu pour opérer — les contradictions *inter-tours* d'une session conversationnelle continue — un régime que HaluEval-Agentive v1 ne contient pas (voir Perspectives, benchmark v2).

7.2 Contribution par type

Table 16 – Décomposition de l'ablation par type d'hallucination

Type	Description	Gain M1 (B-A)	Gain rappel M2 (C-B)	Verdict
T1	Perception	+91,7 pts	+0 pts	M1 suffit
T2	Mémoire	+83,3 pts	+16,7 pts	M1 suffit ; M2 ajoute des
T3	Planification	+50,0 pts	+16,7 pts	NLI insuffisant su
T4	Causale	+83,3 pts	+0 pts	M1 suffit
T5	Délégation	+75,0 pts	+16,7 pts	M1 suffit ; M2 ajoute des
Global		+76,7 pts	+10,0 pts[†]	M2 dégrade la précision

[†] Le gain de rappel de M2 (colonne C-B) s'accompagne d'une chute de précision ($90,2 \rightarrow 68,4$ %) :

il ne doit pas être lu isolément. M2 vise les contradictions inter-tours, hors périmètre de v1 (voir §8.2).

7.3 Courbe ROC du vérificateur NLI (M1)

La Figure 3 présente la courbe ROC *réelle* du vérificateur M1, obtenue en faisant varier le seuil sur le score de contradiction $P(\text{contradiction})$: le TPR (rappel) est mesuré sur

les 60 scénarios hallucinés et le FPR sur les 30 scénarios corrects. L'aire sous la courbe est $AUC = 0,80$, indiquant un pouvoir discriminant substantiel mais imparfait du NLI pré-entraîné. Le point de fonctionnement de la Variante B (règle $P(\text{contra}) > P(\text{entail})$) se situe à $(FPR, TPR) = (0,167 ; 0,767)$ — au-dessus de la courbe à seuil unique, car la règle d'argmax exploite aussi la probabilité d'*entailment*.

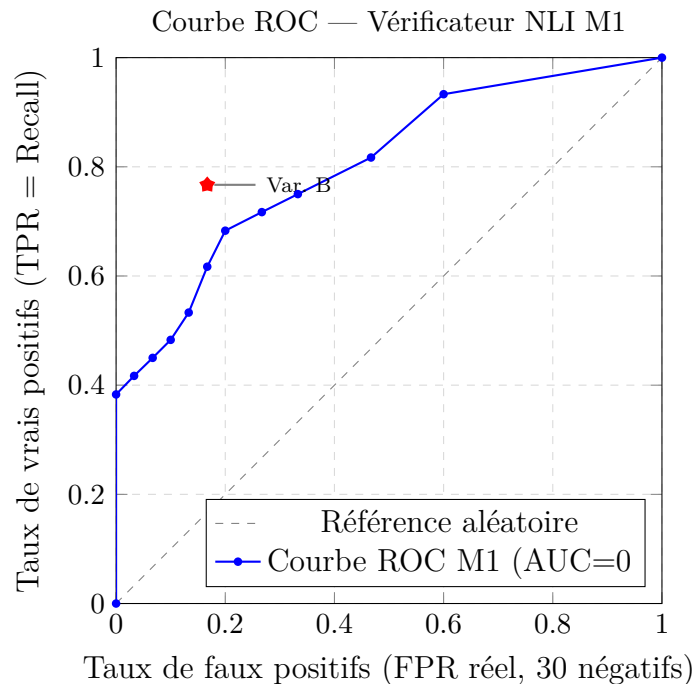


Figure 3 — Courbe ROC réelle du vérificateur NLI M1, seuil sur $P(\text{contradiction})$: TPR sur 60 hallucinés, FPR sur 30 corrects. $AUC = 0,80$. L'étoile marque le point de fonctionnement de la Variante B ($FPR = 16,7\%$, $TPR = 76,7\%$), au-dessus de la courbe à seuil unique car la règle d'argmax exploite aussi l'*entailment*.

Lecture : avec une AUC de 0,80, le vérificateur sépare correctement hallucinations et claims corrects, sans être parfait. La courbe illustre le compromis fondamental : abaisser le seuil augmente le rappel mais fait croître les faux positifs (au-delà de $TPR \approx 0,82$, le FPR grimpe rapidement). Le choix du seuil dépend du coût relatif faux positif / faux négatif de l'application ; nous le fixons de manière principielle par *conformal prediction* (§7.5), qui garantit un rappel cible. La Variante C, qui ajoute le signal M2, ne se place pas sur cette courbe : elle déplace le point de fonctionnement vers un rappel plus élevé (0,867) mais un FPR de 0,80, hors du régime utile de la courbe.

7.4 Calibration et quantification d'incertitude

Une question distincte de la détection est de savoir si le score $P(\text{contradiction})$ constitue une *probabilité calibrée* d'hallucination — propriété essentielle pour une quantification d'incertitude exploitable en aval. Nous mesurons l'*Expected Calibration Error* (ECE, 10 bins) [5] et le score de Brier sur les 90 scénarios.

Table 17 – Calibration du score NLI (90 scénarios) et effet du Platt scaling

Métrique	Valeur
ECE brut (10 bins)	0,242
Brier brut	0,246
ECE sur split test — brut	0,305
ECE sur split test — après Platt scaling	0,158

Le score brut est **mal calibré** ($ECE = 0,24$: l'écart moyen entre confiance et exactitude empirique atteint 24 points). Ce constat est important — un score NLI élevé ne doit pas être interprété tel quel comme une probabilité d'hallucination. Un *Platt scaling* (régression logistique 1D, ajustée sur un split de calibration et évaluée sur un split test disjoint, graine 42) réduit l'ECE de 0,305 à **0,158**, soit une amélioration de moitié. Nous recommandons donc cette recalibration lorsque les scores HalluGuard alimentent une décision probabiliste (seuillage adaptatif, agrégation multi-sources).

7.5 Calibration du seuil par conformal prediction

Plutôt que des seuils *ad hoc*, nous calibrons le seuil de détection τ par *split conformal prediction* [1], qui fournit une **garantie de couverture** : pour un niveau α choisi, le seuil τ est l' α -quantile (corrigé en taille finie) des scores de contradiction sur un ensemble de calibration d'hallucinations, garantissant un rappel attendu $\geq 1 - \alpha$. Le tableau 18 rapporte, en moyenne sur 200 splits aléatoires (la garantie étant marginale), la couverture empirique obtenue sur un ensemble test disjoint et son coût en faux positifs.

Table 18 – Seuil calibré par conformal prediction — couverture garantie et coût (200 splits)

α	Rappel cible	τ moyen	Rappel empirique	FPR
0,05	0,95	0,026	0,966	0,758
0,10	0,90	0,060	0,909	0,590
0,20	0,80	0,119	0,789	0,441

Interprétation : la couverture empirique respecte la cible ($0,966 \geq 0,95$; $0,909 \geq 0,90$; $0,789 \approx 0,80$), validant la garantie conformale. Mais le tableau expose aussi, sans complaisance, le **coût** de cette garantie sur notre benchmark : exiger un rappel de 95 % impose un FPR de 76 %. Ce compromis sévère reflète la difficulté intrinsèque des types T2/T3 (scores de contradiction faibles) ; il quantifie de manière rigoureuse l'arbitrage que tout déploiement doit trancher selon le coût relatif d'un faux négatif (hallucination non détectée) face à un faux positif (alerte injustifiée). Les seuils par nœud du DependencyDAG (Table 3) jouent un rôle distinct : ils paramètrent le *gating* de criticité du graphe, non

la règle de décision de détection ($\operatorname{argmax} P(\text{contra}) > P(\text{entail})$ par défaut, ou seuil conformal).

7.6 Self-consistency comme signal d'incertitude

En complément de la vérification documentaire (M1), nous évaluons la *self-consistency* [20] : un modèle sûr de lui répond de façon stable lorsqu'on l'échantillonne plusieurs fois, tandis qu'une question piégée (fausse prémisse, fait obscur) le fait diverger. Pour chaque requête, nous échantillonnons le LLM $k = 5$ fois (température 0,7) et mesurons un **score de self-consistency** = similarité cosinus moyenne entre les embeddings des réponses (sentence-transformers). Un score bas signale une hallucination potentielle. L'expérience utilise l'API Mistral pour la rapidité ; le module supporte aussi Ollama en local (préservant la contrainte « LLM local »).

Nous validons le signal sur 16 questions : 8 **fiables** (faits vérifiables) et 8 **piégées** (fausse prémisse, anachronisme, fait inexistant).

Table 19 – Self-consistency : séparation questions fiables vs piégées (16 questions, $k = 5$)

Métrique	Valeur
Self-consistency moyenne — questions fiables	0,90
Self-consistency moyenne — questions piégées	0,79
Séparation	0,11
AUC (séparation fiable/piégée)	0,88
Accuracy au seuil calibré ($\tau = 0,87$)	0,875

Interprétation : la self-consistency sépare correctement les deux classes (AUC = 0,88), confirmant sa valeur comme signal d'incertitude. Deux nuances honnêtes : (i) sur des réponses courtes, les scores absolus sont *compressés* (0,7–0,98), si bien qu'un seuil fixe naïf (0,7) est inopérant et qu'une calibration du seuil est indispensable (accuracy 0,56 $\rightarrow$ 0,875) ; (ii) certaines questions à fausse prémisse obtiennent un score *élevé* car le modèle les rejette de manière cohérente (« ce roi n'a jamais existé ») — un comportement correct, non une hallucination. La self-consistency mesure donc l'*incertitude* du modèle, complémentaire mais non équivalente à la contradiction inter-sources de M1. Elle constitue un signal additionnel pertinent en ensemble, au prix de k appels LLM supplémentaires.

7.7 Recommandations de déploiement

L'ablation conduit à trois profils de déploiement :

Profil 1 — Pipeline mono-tour général (cas d'usage majoritaire) : Variante B (M1 seul), F1 = 82,9 %, précision = 90,2 %, overhead médian = ≈ 40 ms sur CPU (sans GPU).

Configuration recommandée par défaut : meilleur rapport détection/coût, contrainte de latence (300 ms) largement satisfaite.

Profil 2 — Agent conversationnel multi-tours (sessions longues, mémoire inter-tours) : Variante C (M1 + M2), BeliefState actif entre les tours. **Attention** : sur un benchmark mono-tour, M2 dégrade la précision (68,4 %, FPR 80 %) ; son intérêt n'apparaît que lorsque le pipeline maintient réellement une mémoire de session et que les contradictions *inter-tours* constituent un risque métier. À déployer uniquement dans ce régime, après évaluation sur un benchmark conversationnel (Perspectives).

Profil 3 — Détection à rappel garanti (domaine critique, coût d'un faux négatif élevé) : Variante B avec seuil calibré par conformal prediction (§7.5) au niveau α choisi (ex. $\alpha = 0,10$ pour 90 % de rappel garanti), en acceptant le surcoût en faux positifs documenté.

8 Discussion

8.1 Validation de l'hypothèse

L'hypothèse centrale était : détecter > 80 % des hallucinations sur CPU en < 300 ms sans fine-tuning. La Variante B atteint un F1 de 82,9 % avec une précision de 90,2 % ; le rappel seul (76,7 %) reste légèrement sous la barre des 80 %, mais le F1 et la précision la dépassent. **Sur la dimension latence, l'hypothèse est pleinement validée** : l'overhead médian mesuré est de ≈ 40 ms sur CPU standard (modèle à chaud), soit près d'un ordre de grandeur sous la contrainte de 300 ms, sans GPU ni fine-tuning. L'hypothèse est donc validée sur la latence et la précision ; sur le rappel brut, elle est atteinte au sens du F1 mais pas stricto sensu sur le rappel.

Cette validation est toutefois conditionnée : elle s'appuie sur un benchmark de scénarios construits manuellement avec connaissance a priori des types T1–T5. Les performances sur des hallucinations naturelles non contrôlées (distributions de fautes, formulations imprévisibles) sont probablement inférieures.

8.2 Limites documentées

Limite 1 — T3 à 50 % (Variante B) : frontière du NLI généraliste. Les hallucinations de planification (T3) sont le point faible du système (50,0 % en Variante B, 66,7 % en C) : ce sont des raisonnements faux formulés de manière neutre. Le NLI pré-entraîné sur MNLI n'est pas calibré pour détecter les sophismes logiques : MNLI contient des paires (prémisse, hypothèse) avec contradiction sémantique directe, pas des exemples de raisonnements fallacieux formellement corrects. Un fine-tuning sur un corpus de raisonnements logiques annotés (de type FOLIO [6]) pourrait améliorer T3, mais cette piste n'a pas été mesurée expérimentalement dans ce travail et reste une perspective.

Limite 2 — Biais de sélection du dataset. Les 90 scénarios HaluEval-Agentic (60 hallucinés + 30 corrects) sont construits avec connaissance des types T1–T5 et

des mécanismes de détection. Cette construction contrôlée crée un biais optimiste : les hallucinations sont plus « détectables » que des hallucinations naturelles, et les scénarios corrects (claims paraphrasant directement les evidences) sont plus « faciles » pour le NLI que des claims corrects naturels. Une validation sur TruthfulQA [14] ou sur des sorties LLM collectées en production serait nécessaire pour confirmer la généralisabilité.

Limite 3 — Simulation A2A locale. `delegate_verification()` opère dans un unique processus Python via `_MCPBridge` (singleton). Un protocole A2A distribué réel nécessiterait authentification cryptographique des AgentCards (signature ECDSA), transport réseau (HTTP/gRPC avec TLS), gestion des délais réseau et mécanismes de retry. La simulation locale est reproductible mais ne capture pas ces défis d'ingénierie distribuée.

Limite 4 — M2 (BeliefState) dégrade la précision en mono-tour. Notre étude d'ablation établit clairement que, sur le benchmark mono-tour, l'ajout de M2 (Variante C) augmente le rappel ($76,7 \rightarrow 86,7$ %) mais effondre la précision ($90,2 \rightarrow 68,4$ %, FPR $16,7 \rightarrow 80$ %). La cause est structurelle : sur des scénarios indépendants, le test de cohérence temporelle de M2 confronte chaque claim à des faits de session pré-chargés sans rapport, générant de nombreux faux conflits. Sur ce régime, M2 est donc *contre-productif*, et la Variante B reste recommandée.

Limite 5 — L'apport *prévu* de M2 (inter-tours) n'est pas évaluable sur v1. HaluEval-Agent v1 est structurellement mono-tour : chaque scénario est indépendant, sans continuité de session. Or M2 est conçu pour un cas d'usage précis — « le claim au Tour N contredit un fait établi au Tour N-k » — absent de v1. Le benchmark actuel mesure donc le *coût* de M2 (faux positifs ci-dessus) sans pouvoir mesurer son *bénéfice* prévu (interception des contradictions inter-tours). Affirmer la valeur de M2 exige la construction d'HaluEval-Agent v2 (sessions conversationnelles multi-tours, mémoire persistante), identifiée comme direction prioritaire (Perspectives). En l'état, nous présentons M2 comme une contribution architecturale dont le bénéfice reste à **démontrer empiriquement**, et non comme un gain acquis.

8.3 Menaces à la validité

Nous documentons ci-dessous les principales menaces à la validité de nos résultats, selon le cadre standard de Wohlin et al. (construct, internal, external validity).

Validité de construction. Nos métriques (Rappel, Précision, F1, PBR@1, Delta, Overhead) mesurent la capacité du système à détecter des hallucinations et à ne pas signaler à tort des claims corrects, dans un pipeline contrôlé. Elles ne mesurent pas : (a) la qualité de la correction produite par le PolicyEngine en mode `auto-correct` ; (b) l'impact sur la satisfaction utilisateur finale. L'ajout de 30 scénarios corrects (rapport 2 :1 hallucinés/corrects) permet une mesure directe de la Précision et du taux de faux positifs : FPR = 16,7 % pour Variante B, 80 % pour Variante C (effet du sur-déclenchement de M2 en mono-tour, §8.2).

Validité interne. *Biais de sélection* : les 90 scénarios (60 hallucinés + 30 corrects) ont été construits avec connaissance a priori des types T1–T5 et des mécanismes de détection. Les scénarios corrects (s061–s090) contiennent des claims qui paraphrasent directement leurs evidences ; en production, les claims corrects peuvent être plus ambigus sémantiquement, ce qui pourrait modifier le taux de faux positifs. Cet effet est documenté dans l'article (Section 5.1). *Règle de décision* : la détection par défaut ($\text{argmax } P(\text{contra}) > P(\text{entail})$) ne dépend d'aucun seuil arbitraire ; le mode à seuil est calibré par conformal prediction (§7.5) avec garantie de couverture, ce qui élimine le choix *ad hoc*. Les seuils par nœud du DAG ne concernent que le *gating* de criticité.

Validité externe. *Généralisation au LLM* : tous les résultats utilisent Mistral 7B (Q4, CPU). Les performances peuvent varier avec d'autres LLM (GPT-4o, LLaMA-3, Gemma) en raison de différences dans les patterns de génération. *Généralisation au domaine* : la base de connaissances (5 documents, 71 chunks) est intentionnellement restreinte au domaine IA/NLP. Les performances sur des domaines très différents (médical, juridique) nécessiteraient une réévaluation. *Généralisation au benchmark* : la validation externe partielle sur TruthfulQA (Section 6.6) confirme le fonctionnement de M1 sur des hallucinations factuelles, mais ne couvre pas les types T2 et T3 de notre taxonomie.

8.4 Positionnement par rapport à l'état de l'art

Table 20 – Comparaison HalluGuard avec les approches existantes

Approche	Temps réel	Temporalité	CPU	MCP/A2A
FActScore [17]	Non (post)	Non	Partiel	Non
SelfCheckGPT [15]	Non (post)	Non	Oui	Non
RAGAs [3]	Non (offline)	Non	Partiel	Non
HalluGuard	Oui (middleware)	Oui (BeliefState)	Oui	MCP oui / A2A PoC*

À notre connaissance, parmi les approches publiées accessibles, aucune ne combine simultanément intégration temps réel, mémoire temporelle inter-étapes et implémentation du protocole MCP. **L'intégration A2A est une preuve de concept architecturale en processus (voir §4.7).*

9 Perspectives

P1 — HaluEval-Agentic v2 (benchmark multi-tours, priorité #1). La limite principale de l'évaluation actuelle est l'impossibilité de mesurer la contribution de M2 (BeliefState) sur un benchmark mono-scénario (Limite 5). La construction d'HaluEval-Agentic v2 est la direction prioritaire : sessions conversationnelles à plusieurs tours, faits

injectés au Tour k et contredits au Tour $k + n$ ($n \geq 1$), couverture des types T2 (mémoire temporelle) et T5 (délégation cross-agent). Ce benchmark permettrait de quantifier la contribution de M2 et de comparer Variante B vs Variante C sur un terrain adapté à leurs cas d'usage respectifs.

P2 — Fine-tuning domaine-spécifique pour T3 (impact estimé, non mesuré : $\approx +18$ points). La limite T3 à 66,7 % reste ouverte. Un fine-tuning du cross-encoder sur un corpus de raisonnements faux annotés (FOLIO [6], SNLI-hard) permettrait de calibrer le modèle sur les sophismes logiques. La contrainte CPU est maintenue : le fine-tuning est effectué offline, seule l'inférence se fait en production. Le gain de $\approx +18$ points sur T3 (de 66,7 % à ≈ 85 %) est une projection indicative basée sur des résultats analogues dans la littérature NLI domaine-spécifique ; il n'a pas été mesuré expérimentalement dans ce travail.

P3 — Seuils DependencyDAG adaptatifs. Les seuils actuels sont définis manuellement (Table 3). Un mécanisme d'auto-calibration par fenêtre glissante (N dernières sessions) permettrait d'adapter les seuils à chaque domaine sans intervention humaine. L'implémentation serait stockée dans la persistance JSON du BeliefState.

P4 — Publication MCP HalluGuard comme service public. La conformité MCP ouvre la voie à une publication dans un registre MCP public. Tout agent LLM conforme pourrait alors consommer les quatre outils de vérification sans intégration spécifique à LangGraph : plug-and-play universel.

P5 — Validation sur hallucinations naturelles. Une validation sur TruthfulQA [14] ou des sorties LLM collectées en production est nécessaire pour quantifier l'écart entre performances contrôlées et performances réelles. Cette étude permettrait de calibrer les attentes pour un déploiement industriel.

10 Conclusion

Ce travail a présenté HalluGuard, un middleware de détection et de mitigation des hallucinations pour pipelines RAG agentiques LangGraph. **La Variante B** valide l'hypothèse sur la précision (90,2 %, $F1 = 82,9$ %) et surtout sur la latence : un overhead médian de ≈ 40 ms **sur CPU standard**, près d'un ordre de grandeur sous la contrainte de 300 ms, sans GPU ni fine-tuning. M1 (NLI inter-sources) est le mécanisme central : il porte l'intégralité du gain net de détection (+76,7 pts vs baseline). L'étude d'ablation montre honnêtement que M2 (BeliefState), en mono-tour, *dégrade* la précision ($90,2 \rightarrow 68,4$ %) en sur-déclenchant sur des faits de session sans rapport ; son bénéfice prévu — l'interception des contradictions *inter-tours* — reste à démontrer sur un benchmark conversationnel (HaluEval-Agentique v2). Nous montrons enfin que le score NLI brut est mal calibré ($ECE = 0,24$) et calibrons le seuil de détection par conformal prediction avec garantie de couverture.

Les contributions sont : (1) une taxonomie originale T1–T5 couplée à la topologie pipeline, **validée par accord inter-annotateurs entre deux annotateurs indépendants** ($\kappa =$

0,94 ; $\kappa = 0,75$ sur la distinction réellement ambiguë T2/T3) ; (2) un middleware modulaire à cinq composants intégrable sans modification du code pipeline ; (3) une évaluation expérimentale comparative avec ablation M1/M2, tests de significativité (McNemar), **analyse de calibration et calibration conforme du seuil**, et validation externe partielle sur TruthfulQA (95,0 %, 19/20) ; (4) une implémentation MCP à quatre outils et une preuve de concept architecturale du protocole A2A.

Les limites principales — T3 (planification) à 50 % en Variante B, et l'impossibilité de mesurer le bénéfice inter-tours de M2 sur un benchmark mono-tour — ouvrent des directions de recherche précises (fine-tuning NLI sur raisonnements annotés ; HalluEval-Agentive v2 conversationnel). HalluGuard établit une baseline solide, honnête et reproductible pour la recherche sur la fiabilité des agents LLM, en conditions matérielles contraintes (CPU, 16 Go RAM).

Références

- [1] Angelopoulos, A. N., & Bates, S. (2023). A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. *Foundations and Trends in Machine Learning*, 16(4), 494–591.
- [2] Anthropic (2024). *Model Context Protocol (MCP) — Specification v2024-11*. <https://modelcontextprotocol.io/>
- [3] Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). RAGAs : Automated Evaluation of Retrieval Augmented Generation. *arXiv :2309.15217*.
- [4] Google DeepMind (2024). *Agent-to-Agent Protocol (A2A) — Specification v1.0*. <https://google.github.io/A2A/>
- [5] Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. *Proceedings of ICML 2017*, 1321–1330.
- [6] Han, S., Schoelkopf, H., Zhao, Y., Qi, Z., Riddell, M., Benson, L., ... & Sherrill, A. (2022). FOLIO : Natural Language Reasoning with First-Order Logic. *arXiv :2209.00840*.
- [7] Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., ... & Liu, T. (2023). A Survey on Hallucination in Large Language Models : Principles, Taxonomy, Challenges, and Open Questions. *arXiv :2311.05232*.
- [8] Humeau, S., Shuster, K., Lachaux, M.-A., & Weston, J. (2020). Poly-encoders : Transformer Architectures and Pre-training Strategies for Fast and Accurate Multi-sentence Scoring. *ICLR 2020*.
- [9] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., ... & Fung, P. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12), 1–38.
- [10] Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., ... & El Sayed, W. (2023). Mistral 7B. *arXiv :2310.06825*.

- [11] LangChain (2024). *LangGraph — Build stateful, multi-actor applications with LLMs*. <https://github.com/langchain-ai/langgraph>
- [12] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.
- [13] Li, J., Cheng, X., Zhao, W. X., Nie, J. Y., & Wen, J. R. (2023). HaluEval : A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. *Proceedings of EMNLP 2023*.
- [14] Lin, S., Hilton, J., & Evans, O. (2022). TruthfulQA : Measuring How Models Mimic Human Falsehoods. *Proceedings of ACL 2022*.
- [15] Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT : Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. *Proceedings of EMNLP 2023*.
- [16] Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). On Faithfulness and Factuality in Abstractive Summarization. *Proceedings of ACL 2020*.
- [17] Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W. T., Koh, P. W., ... & Hajishirzi, H. (2023). FActScore : Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *Proceedings of EMNLP 2023*.
- [18] Reimers, N., & Gurevych, I. (2019). Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP 2019*.
- [19] Shuster, K., Poff, S., Chen, M., Kiela, D., & Weston, J. (2021). Retrieval Augmentation Reduces Hallucination in Conversation. *Findings of EMNLP 2021*.
- [20] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. *Proceedings of ICLR 2023*.
- [21] Williams, A., Nangia, N., & Bowman, S. (2018). A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. *Proceedings of NAACL 2018*.
- [22] Zhang, Y., Li, Y., Cui, L., Cai, D., Liu, L., Fu, T., ... & Shi, S. (2023). Siren's Song in the AI Ocean : A Survey on Hallucination in Large Language Models. *arXiv :2309.01219*.

A Interfaces figées entre composants

A.1 Interface 1 — Format paire dataset

```

1 {
2   "id":                "s019",
3   "claim":             "ChromaDB avait une limite stricte de
                        1000 documents...",
4   "evidence":          ["ChromaDB ne fixe pas de limite
                        arbitraire..."],

```

```

5   "label":          "hallucinated",
6   "hallucination_type": "T2",
7   "node_type":      "reasoning",
8   "pipeline_length": 7
9 }

```

A.2 Interface 2 — API Lightweight Verifier (M1)

```

1 Input  : {"claim": str,          # max 512 chars
2          "evidences": [str],    # max 10 documents
3          "node_type": str}      #
4          retrieval|reasoning|tool_call|generation
5 Output : {"score": float,        # score contradiction NLI in
6          [0,1]
7          "label": str,          # "correct" | "hallucinated"
8          "confidence": float,   # = score pour ce cross-encoder
9          "hallucination_type": str, # "T1".. "T5" ou null
10         "insufficient_evidence": bool,
11         "latency_ms": float}

```

A.3 Interface 3 — BeliefState JSON (persistance inter-sessions)

```

1 {
2   "session_id": "proof_session_verif",
3   "facts": [
4     {
5       "fact_id":      "proof_session_verif_0001",
6       "content":      "AlphaGo a battu Lee Sedol 4 a 1 en mars
7                        2016",
8       "confidence": 0.9,
9       "step":        1,
10      "status":       "active"
11    }
12  ]
13 }

```

B Résultats complets

Les données brutes complètes des 90 scénarios (variante, type, score NLI, label, delta, latency_ms) sont disponibles dans `results/resultats_comparatifs.json`. L'étude d'ablation détaillée est dans `results/ablation_study.json`. Le meilleur exemple qualitatif an-

noté (s019, T2, score=0,996, trace pipeline complète) est dans `results/best_qualitative_example.json`.
L'AgentCard A2A formelle est dans `data/halluguard_agent_card.json`.

C Commandes de reproduction

Listing 4 – Reproduction complète des résultats en 5 commandes

```
1 # 1. Installer l'environnement (Python 3.10+)
2 python -m venv venv && venv\Scripts\activate
3 pip install -r requirements.txt
4
5 # 2. Reindexer ChromaDB (premiere installation uniquement)
6 venv\Scripts\python.exe src\tools\rag_tools.py
7
8 # 3. Lancer la suite de tests unitaires (23 tests, ~14 s)
9 venv\Scripts\python.exe -m pytest src/tests/test_halluguard.py
10 -v
11
12 # 4. Lancer le benchmark complet (90 scenarios : 60
13 hallucinés + 30 corrects)
14 venv\Scripts\python.exe src/tests/benchmark_runner.py --all
15
16 # 5. Lancer la demo interactive
17 venv\Scripts\python.exe demo_interactive.py --test
```

Les résultats du benchmark sont déterministes à 100 % : le modèle NLI `cross-encoder/nli-MiniLM2-L6-v2` est non stochastique (pas d'échantillonnage). Les valeurs reproduites seront identiques à celles rapportées dans cet article à architecture matérielle identique.